# Project 1: SIMD Advantage Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 21, 2026

**Abstract**

We measure scalar-versus-SIMD speedup for four single-threaded kernels (SAXPY, dot product, elementwise multiply and a 3-point stencil) on an Intel Core i9-10900F (AVX2+FMA, 256 bit vectors) at a fixed 2.8 GHz. With cache-resident data a `-march=native` build reaches about 5.8× over scalar for SAXPY and the generic SSE build about 3×. In DRAM the gain compresses to 1.88–2.02× for SAXPY: the native kernels sit on the measured bandwidth roof, and what remains is the scalar loop's own instruction cost. Float64 usually keeps about half of float32's cache-resident gain. Any non-unit stride or gather erases the advantage; the AVX2 hardware gather is even 1.5–1.6× slower than scalar in cache. A cache-aware roofline with separately measured ceilings (89.2 GFLOP/s FMA peak, triad bandwidth per level) places L3- and DRAM-resident native kernels at 95 % to 107 % of their roof. Our negative cases are memory-bound SAXPY and the non-reassociated dot product (≈ 1×).

## 1 Introduction

Single-instruction-multiple-data (SIMD) units let one instruction operate on a whole vector of elements. On the Intel Core i9-10900F studied here the widest unit is AVX2 with FMA: a 256 bit register holds 8 float32 or 4 float64 lanes, so the *ideal* speedup over scalar code is 8× or 4×. Loads and stores, loop tails, reduction dependencies and, above all, memory bandwidth keep real kernels below that. We measure where the benefit appears, how far it falls short of the lane count, and when it disappears:

1. How much faster are auto-vectorized builds than scalar builds for each kernel, and how does that change from cache-resident to DRAM-resident working sets? (Sections 5 and 6)
2. What do alignment, loop tails, stride and gather access, and data type cost? (Sections 7 to 9)
3. Where do these kernels sit in a roofline model of this machine? (Section 10)
4. When does the expected SIMD benefit fail to appear? (Section 11)

## 2 Methodology

### 2.1 Platform

This Comet Lake core implements AVX2 and FMA but *not* AVX-512, so the widest vector is 256 bit (`ymm`). `-march=native` targets AVX2+FMA; the generic `x86-64` target only guarantees SSE2, so the plain auto-vectorized build emits 128 bit (`xmm`) code. That gap is why `autovec` and `native` are separate variants (Table 3).

Table 1: Measurement environment.

| | |
|---|---|
| CPU | Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64) |
| SIMD ISA | sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm) |
| Topology | 1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s) |
| SMT | off |
| Caches | L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance) |
| Memory | 63 GiB total; DIMMs: 2 × 32 GiB DDR4-3200, one per channel (A, B), running at 2933 MT/s (dmidecode) |
| Pages | base 4096 bytes; THP: always |
| OS / kernel | Arch Linux; 7.2.9-arch1-1 |
| Frequency policy | governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance |
| Pinning / isolation | Each run is pinned to one physical core (`--cpu 2`, `sched_setaffinity`); SMT is disabled, so the core has no sibling thread, and no other pinning/isolation is used.; no isolcpus |
| Compilers | gcc (GCC) 16.2.1 20260810; clang version 23.1.3 |
| Tools | perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present |
| GPU (optional) | NVIDIA GeForce RTX 3070; CUDA not available (no CUDA) |
| Env. snapshot | 2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`) |

Table 2: Kernels and floating-point operations (FLOPs) per processed element.

| Kernel | Definition | FLOPs / element |
|---|---|---|
| SAXPY | $y_i \leftarrow a\,x_i + y_i$ | 2 |
| Dot product | $s \leftarrow \sum_i x_i\,y_i$ | 2 |
| Elementwise multiply | $z_i \leftarrow x_i\,y_i$ | 1 |
| 3-point stencil | $y_i \leftarrow a\,x_{i-1} + b\,x_i + c\,x_{i+1}$ | 5 |

Table 3: Build variants. Only the kernel translation unit (`src/kernels.c`) differs; the timing harness (`main.c`, `common.c`) and the reference (`reference.c`) are compiled identically across variants.

| Variant | Kernel compiler flags (from `simd_bench --info`) |
|---|---|
| scalar | `-O3 -fno-tree-vectorize -fno-tree-slp-vectorize` |
| autovec | `-O3` (generic `x86-64` ⇒ SSE2, 128 bit) |
| native | `-O3 -march=native` (AVX2+FMA, 256 bit) |
| native512 | not available on this CPU (no AVX-512; 256 bit is the widest ISA) |

## 2.2 Kernels

A multiply and an add count as one FLOP each, so an FMA is two. All four kernels are implemented and validated; SAXPY, with the simplest unit-stride access, is the representative kernel for the locality, alignment and stride studies. Inputs are a fixed-seed linear-congruential sequence in $[-0.5, 0.5)$ (no denormals, NaNs or overflow); the gather uses a Fisher–Yates permutation of $[0, N)$. Float32 SAXPY touches 8 B per element, so its working set crosses the 32 KiB L1d near $N \approx 4096$, the 256 KiB L2 near $N \approx 32\,768$ and the 20 MiB L3 near $N \approx 2.6 \times 10^6$; plots mark these boundaries.

## 2.3 Build variants

The compiler is GCC gcc (GCC) 16.2.1 20260810. The reference is built with `-O2 -fno-tree-vectorize -ffp-contract=off`, so it evaluates each expression in strict, non-

fused order. No variant uses `-ffast-math`: results stay reproducible, but the compiler cannot reassociate the dot-product reduction, an effect we measure rather than hide (Sections 4 and 11).

### 2.4 Data collection, timing and hypotheses

All measurements were taken on **2026-10-21** on an idle machine, settings checked before each run and jobs serialized through a lock file: `baseline` at 20:38, the other six experiments from 20:47 to 20:54 (Table 11). Beforehand the user masked an ACPI event storm (GPE 0x42) that had stolen $\approx 13\,\%$ of wall time in the companion Cache & Memory project; the mask does not survive a reboot (Section A). Governor performance, turbo disabled and SMT off (Table 1) fix the core at 2.8 GHz. Two checks guard against a shared core: no configuration varies by more than 1.36× between its fastest and slowest repetition (all but one $\leq 1.32\times$), and the user-space cycle counter, where stolen time would show as a lower clock, reads 2.78 GHz to 2.79 GHz for all 96 locality and 123 ceilings configurations.

- Timer: `CLOCK_MONOTONIC_RAW` (1 ns resolution); each repetition times a calibrated batch of at least 25 ms, so resolution and call overhead are negligible. 3 warm-up invocations, 12 repetitions; median with interquartile-range (IQR) error bars. Speedup error bars span $[\, q_{25}(\text{SIMD})/q_{75}(\text{scalar}),\ q_{75}(\text{SIMD})/q_{25}(\text{scalar}) \,]$ (`scripts/speedup_iqr.awk`).
- Results pass through a `volatile` sink and kernels live in their own translation unit, so no loop is dead-code eliminated. The sweep (`scripts/run.sh`) is not shuffled.
- An earlier turbo-on sweep (2026-10-12, commit `73f8fb8`, raw directory `20261012-182813`) is not used for any result. Its cache-resident rates were 1.72–1.75× the fixed-clock values (dot product 2.43 GFLOP/s to 2.49 GFLOP/s at 0.5 FLOP/cycle, i.e. ≈ 4.9 GHz), while DRAM bandwidth does not scale, so its DRAM-resident SAXPY speedups were only 1.32–1.44×, the single turbo figure we quote.

Each experiment opens with a hypothesis from first principles (lane counts, cache sizes, port counts). Some were drafted after the superseded turbo sweep, so we treat them as predictions and state where the data contradicts them.

### 2.5 Machine ceilings and hardware counters

The roofline (Section 10) needs ceilings *not* derived from the kernels it explains; a separate program (`src/ceilings.c`, `run.sh ceilings`) measures them:

- *Peak FLOP rate*: 12 independent register accumulators $a \leftarrow a\,m + c$ (with 4-cycle latency and two FMA ports, $4 \times 2 = 8$ chains saturate the ports), as scalar `mulss`+`addss` (`scalar`'s ceiling), scalar FMA, SSE `mulps`+`addps` (`autovec`, no FMA) and AVX2 `vfmadd231ps/pd` (`native`).
- *Bandwidth*: STREAM-style *read*, *copy* and *triad* ($a_i = b_i + s\,c_i$) with AVX2 loads/stores, at a footprint inside each level (L1 16 KiB, L2 128 KiB, L3 8 MiB, DRAM 512 MiB) and over a 4 KiB–512 MiB sweep. Roofs and kernel intensities count actual traffic: explicit loads and stores in L1, where stores hit, plus the write-allocate read in L2, L3 and DRAM (Table 10). The read stream's add chain caps it at one 32 B load per cycle (89 GiB/s), so it is never a roof.

*Hardware counters.* The AI agent that ran the benchmarks had no root; the user applied the root-only settings (governor, turbo, SMT, GPE mask, `perf_event_paranoid` = 0). `src/perfctr.c`

opens the thread's own user-space core-PMU events (cycles, instructions, LLC misses, L1D read misses) with `perf_event_open(2)`, unprivileged up to `perf_event_paranoid` = 2, enabled only around each timed repetition (`NA` if unavailable).

# 3 Correctness Validation

Every build variant is checked two independent ways before its timings are trusted (`make test`, `scripts/validate.sh`). *Reference comparison:* `src/reference.c` holds plain scalar loops compiled with `-fno-tree-vectorize -ffp-contract=off` (strict, non-fused, left-to-right order); the driver runs kernel and reference on identical inputs and compares them element by element (or the scalar result, for the dot product) before timing.

| | |
|---|---|
| **Cases** | kernels × {float32, float64}; $N$ around multiples of the lane count $(1, 2, 3, 7, 8, 9, 15, 16, 17, 31, 33, 64, 127, 1000, 1024, 4095, 4096, 4097, 65536)$; start offsets 4, 8, 12, 16, 20, 28, 32 and 60 B; strides $2, 4, 8, 16$; gather; explicit AVX2 gather (native only, $N \in \{7, 8, 9, 16, 17, 1000, 4096, 65536\}$); int32 multiply |
| **Tolerance** | relative error $\leq 10^{-5}$ (float32) and $\leq 10^{-12}$ (float64): loose enough for FMA contraction in the vectorized builds, tight enough to catch real bugs |
| **Command** | `make test` |

*Known-answer self-test* (`simd_bench --selftest`): fixed inputs with known results for every kernel and the AVX2 gather (e.g. $y = 2x + y$ with $x = 1, y = 3$ gives 5), which guards against kernel and reference sharing a mistake. Remainder sizes and misaligned starts exercise the epilogue and prologue that later experiments stress.

Result (`data/raw/validation/20261021-231629/make_test.txt`): all 578 cases pass (190 each for scalar and autovec, 198 for native, which adds the AVX2 gather). The largest relative error is $2.1 \times 10^{-7}$ (native float32 dot product, $N = 127$); every other case, including all float64 and int32 cases, matches bit for bit. The test coefficients ($a = 0.5$; stencil $0.25, 0.5, 0.25$) make every product exact, so only the reduction exercises the tolerance.

# 4 Vectorization Verification

**Hypothesis (from first principles, then compared with the data)**

The `scalar` build should emit only scalar SSE (`ss`/`sd`) instructions and report zero vectorized loops. `autovec`, limited to the generic SSE2 target, should vectorize the streaming kernels at 128 bit (4 float32 lanes). `native` should vectorize them at 256 bit (`ymm`, 8 float32 / 4 float64 lanes) and fuse the multiply–add in SAXPY and the stencil into `vfmadd`. We expect the compiler to *refuse* to vectorize the dot-product reduction usefully, because summation reassociation is illegal without `-ffast-math`.

Table 4 counts, per kernel function and variant, the instructions that use vector registers or packed operations (`objdump`, `make report`). The few matches in the scalar column are register-clearing `pxor`/`movaps`; GCC reports *zero* vectorized loops for `scalar`.

## 4.1 Compiler report

`make report` (`data/raw/vectorization/20261021-001032`) recompiles the unchanged `src/kernels.c` with each variant's flags; `objdump` confirms the kernels match the timed binaries instruction for instruction. `-fopt-info-vec` for SAXPY (`kernels.c:29`):

Table 4: SIMD instructions per kernel function and build variant.

| Build | Function | Insns | SIMD | Widest |
|---|---|---|---|---|
| scalar | saxpy_f32 | 15 | 1 | xmm(128) |
| scalar | dot_f32 | 19 | 4 | xmm(128) |
| scalar | mul_f32 | 12 | 0 | scalar |
| scalar | stencil_f32 | 34 | 1 | xmm(128) |
| scalar | saxpy_f64 | 14 | 1 | xmm(128) |
| scalar | dot_f64 | 19 | 4 | xmm(128) |
| scalar | mul_f64 | 12 | 0 | scalar |
| scalar | stencil_f64 | 31 | 1 | xmm(128) |
| scalar | mul_i32 | 13 | 0 | scalar |
| scalar | saxpy_f32_stride | 31 | 1 | xmm(128) |
| scalar | saxpy_f32_gather | 15 | 1 | xmm(128) |
| scalar | saxpy_f32_gather_avx2 | 14 | 1 | xmm(128) |

| Build | Function | Insns | SIMD | Widest |
|---|---|---|---|---|
| autovec | saxpy_f32 | 67 | 11 | xmm(128) |
| autovec | dot_f32 | 58 | 11 | xmm(128) |
| autovec | mul_f32 | 63 | 6 | xmm(128) |
| autovec | stencil_f32 | 121 | 25 | xmm(128) |
| autovec | saxpy_f64 | 48 | 7 | xmm(128) |
| autovec | dot_f64 | 42 | 7 | xmm(128) |
| autovec | mul_f64 | 48 | 4 | xmm(128) |
| autovec | stencil_f64 | 78 | 16 | xmm(128) |
| autovec | mul_i32 | 69 | 4 | xmm(128) |
| autovec | saxpy_f32_stride | 84 | 12 | xmm(128) |
| autovec | saxpy_f32_gather | 14 | 1 | xmm(128) |
| autovec | saxpy_f32_gather_avx2 | 14 | 1 | xmm(128) |

| Build | Function | Insns | SIMD | Widest |
|---|---|---|---|---|
| native | saxpy_f32 | 78 | 11 | ymm(256) |
| native | dot_f32 | 78 | 19 | ymm(256) |
| native | mul_f32 | 79 | 9 | ymm(256) |
| native | stencil_f32 | 110 | 16 | ymm(256) |
| native | saxpy_f64 | 60 | 11 | ymm(256) |
| native | dot_f64 | 57 | 13 | ymm(256) |
| native | mul_f64 | 69 | 9 | ymm(256) |
| native | stencil_f64 | 88 | 16 | ymm(256) |
| native | mul_i32 | 80 | 9 | ymm(256) |
| native | saxpy_f32_stride | 93 | 12 | ymm(256) |
| native | saxpy_f32_gather | 14 | 1 | xmm(128) |
| native | saxpy_f32_gather_avx2 | 95 | 18 | ymm(256) |

```
# native  (-O3 -march=native):
src/kernels.c:29:26: optimized: loop vectorized using 32 byte vectors and unroll factor 8
src/kernels.c:29:26: optimized: epilogue loop vectorized using 16 byte vectors and unroll factor 4
# autovec (-O3, generic x86-64):
src/kernels.c:29:26: optimized: loop vectorized using 16 byte vectors and unroll factor 4
src/kernels.c:29:26: optimized: epilogue loop vectorized using 8 byte vectors and unroll factor 2
# scalar  (-O3 -fno-tree-vectorize): 0 loops vectorized
```

32 B vectors are 8 float32 lanes (native), 16 B are 4 (autovec); GCC's "unroll factor" is the vectorization factor, not extra unrolling. The native epilogue is one 128 bit step (if 4 or more elements remain) plus up to three scalar iterations; autovec's is one 8 B step plus at most one scalar iteration. The loop is *versioned* on a run-time aliasing check between `x` and `y`, and GCC does not peel iterations to reach an aligned start.

## 4.2 Disassembly

The native SAXPY inner loop is one fused multiply–add on `ymm` registers:

```
vmovups   (%rdi,%rax,1),%ymm0           # load 8 x[i]
vfmadd213ps (%rsi,%rax,1),%ymm1,%ymm0   # ymm0 = a*x + y   (one FMA, 8 lanes)
vmovups   %ymm0,(%rsi,%rax,1)           # store 8 y[i]
```

The scalar build handles one element at a time and cannot fuse, because the generic `x86-64` target lacks FMA:

```
movss (%rdi,%rax,4),%xmm0  # x[i]
mulss %xmm1,%xmm0          # a*x[i]
addss (%rsi,%rax,4),%xmm0  # + y[i]   (separate mul + add: generic x86-64 has no FMA)
movss %xmm0,(%rsi,%rax,4)  # y[i]
```

The dot product is the exception: native multiplies eight lanes at once but sums them one lane at a time, in source order:

```
vmulps (%rdi,%rax,1),%ymm1,%ymm1  # 8 products at once ...
vaddss %xmm1,%xmm0,%xmm0          # ... but summed one lane at a time,
vaddss %xmm3,%xmm0,%xmm0          #     in strict source order
...
```

### 4.3 Stride and gather code

`saxpy_f32_stride` shows `ymm` instructions in `native` only because GCC versions the loop on `s == 1`; for $s > 1$ it runs a scalar `vfmadd213ss` loop (`mulss`/`addss` in the other builds). No build vectorizes the plain gather ("no vectype for stmt"), so we added an explicit AVX2 version, `saxpy_f32_gather_avx2`, compiled only into `native`:

```
vmovdqu    (%rax),%ymm1                # 8 indices idx[i..i+7]
vgatherdps %ymm5,(%rdi,%ymm1,4),%ymm0  # gather 8 x[idx]
vgatherdps %ymm7,(%rsi,%ymm1,4),%ymm3  # gather 8 y[idx]
vfmadd132ps %ymm4,%ymm3,%ymm0          # one 8-lane FMA
vmovss     %xmm0,(%rsi,%rdx,4)         # ... then 8 scalar stores:
vextractps $0x1,%xmm0,(%rsi,%rdx,4)    #     AVX2 has no scatter
```

The loads are SIMD; the stores cannot be, since AVX2 has no scatter.

### 4.4 Interpretation

The evidence matches the hypothesis: `native` uses `ymm` for SAXPY, multiply and the stencil (with FMA in SAXPY and the stencil), `autovec` uses `xmm`, and `scalar` is genuinely scalar. Tails take a narrower vector step plus a scalar remainder; GCC does not use AVX2's masked `vmaskmovps` (no AVX-512 masks here). The dot product's packed multiplies feed a latency-bound scalar accumulator, so "vectorized" does not mean "fast" (Section 11).

## 5 Scalar vs. SIMD Baseline

**Hypothesis (from first principles, then compared with the data)**

For compute-friendly, unit-stride kernels (SAXPY, elementwise multiply, stencil) we expect the `native` speedup to approach the float32 lane count (8×) while data is L1/L2-resident, and to fall toward 1× once the working set is DRAM-resident and all variants share the same bandwidth. `autovec` (4 lanes) should track at roughly half the `native` speedup. The dot product should barely speed up at any size because its reduction does not reassociate (Section 4).

| | |
|---|---|
| **Fixed** | float32; unit stride; aligned arrays; one thread pinned to CPU 2 |
| **Varied** | kernel; variant (scalar, autovec, native); $N = 2^8 \dots 2^{25}$ |
| **Repetitions** | 12 per point; median, IQR error bars |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh baseline` |

### 5.1 Analysis

At $N = 4096$ (L1-resident) the `native` speedups are SAXPY 5.79× (20.2 GFLOP/s), elementwise multiply 3.55×, stencil 3.49× (17.3 GFLOP/s) and dot 1.01×; `autovec` reaches 3.02×, 2.54× and 2.79× for the first three. None reaches 8× even in cache: native SAXPY needs 0.22 to 0.28 cycles per element in L1 (Section 6) against the 0.125 cycles that two load ports and one store port would allow, so load/store issue, not FLOPs, limits it.

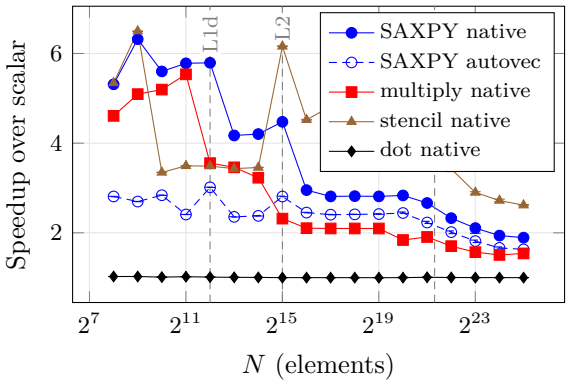



Figure 1: Speedup of the SIMD builds over scalar (median; error bars: range implied by the two IQRs). Gains peak in cache and compress in DRAM; the dot product never benefits.

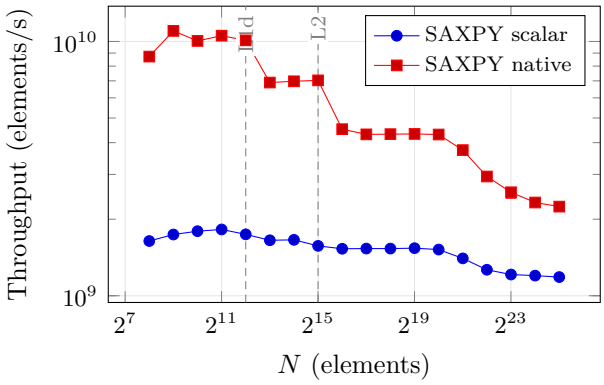



Figure 2: Achieved SAXPY throughput (median, IQR). Native drops to a DRAM-limited plateau; scalar loses comparatively little outside cache.

Table 5: Achieved GFLOP/s (median) of every kernel and build at one $N$ per level: L1 ($N = 1024$), L2 (16 384), L3 (524 288) and DRAM (16 777 216). The same points appear in the roofline (Figure 10).

| Kernel | Build | L1 | L2 | L3 | DRAM |
|---|---|---|---|---|---|
| SAXPY | scalar | 3.59 | 3.32 | 3.08 | 2.40 |
| | autovec | 10.19 | 7.89 | 7.44 | 3.99 |
| | native | 20.10 | 13.94 | 8.66 | 4.65 |
| dot | scalar | 1.41 | 1.39 | 1.39 | 1.33 |
| | autovec | 1.41 | 1.39 | 1.39 | 1.33 |
| | native | 1.42 | 1.39 | 1.39 | 1.33 |
| multiply | scalar | 1.78 | 1.65 | 1.56 | 1.08 |
| | autovec | 6.30 | 3.80 | 3.14 | 1.53 |
| | native | 9.25 | 5.32 | 3.26 | 1.63 |
| stencil | scalar | 4.95 | 4.89 | 4.34 | 4.03 |
| | autovec | 13.59 | 11.83 | 13.12 | 8.72 |
| | native | 16.57 | 16.91 | 20.67 | 10.95 |

*Elementwise multiply* streams three arrays (12 B/element) and leaves L1 earlier ($N \approx 2700$); its speedup peaks at 5.54× at $N = 2048$. The *stencil* does the most work per element (5 FLOP, reusing $x$) and keeps a 2.6–2.7× advantage even in DRAM. Its cache-resident speedup is irregular: 3.3–3.5× for $N = 1024 \ldots 16\,384$ but 4.5–6.2× for $N = 32\,768 \ldots 1\,048\,576$ (Section 12). The *dot product*, the reduction case of Section 4, stays at $\approx 1\times$ (1.33 GFLOP/s to 1.49 GFLOP/s) at every size.

By $N = 16\,777\,216$ (128 MiB) every speedup has compressed (Figure 1 and Table 5). Figure 2 shows the asymmetry: native SAXPY falls from 20.2 to 4.7 GFLOP/s, scalar only from about 3.5 to 2.4 GFLOP/s, because scalar SAXPY stays largely instruction-bound even in DRAM (Section 6). Other sweeps give 1.88–2.02× for SAXPY at $N = 2^{24}$–$2^{25}$.

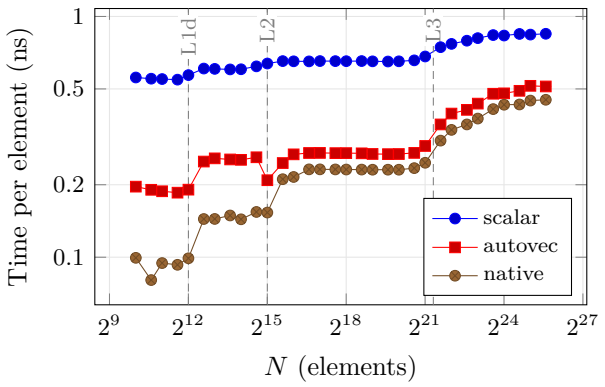



Figure 3: SAXPY time per element across cache levels: native rises in steps at each boundary.

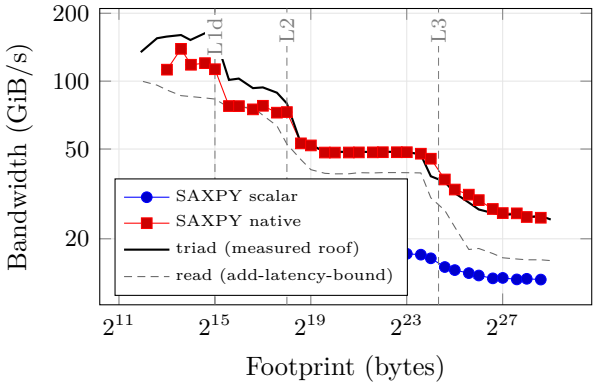



Figure 4: Effective bandwidth versus footprint: SAXPY against the independently measured triad (explicit traffic below 32 KiB, incl. write-allocate above). Where native meets the triad it is bandwidth-bound. The dashed read stream is add-latency-bound ($\approx$ 89 GiB/s in L1 and L2), not a ceiling.

Table 6: Effective SAXPY bandwidth (GiB/s, 12 B/element) at one $N$ per level, next to that level's independently measured triad roof (Section 2.5).

| Level | $N$ | Footprint | scalar | autovec | native | triad roof |
|---|---|---|---|---|---|---|
| L1 | 2048 | 16 KiB | 20.3 | 59.5 | 118.2 | 155.6 |
| L2 | 16384 | 128 KiB | 18.5 | 44.0 | 77.8 | 90.5 |
| L3 | 524288 | 4 MiB | 17.2 | 41.6 | 48.4 | 48.6 |
| DRAM | 33554432 | 256 MiB | 13.3 | 21.7 | 25.0 | 24.3 |

# 6 Locality Sweep

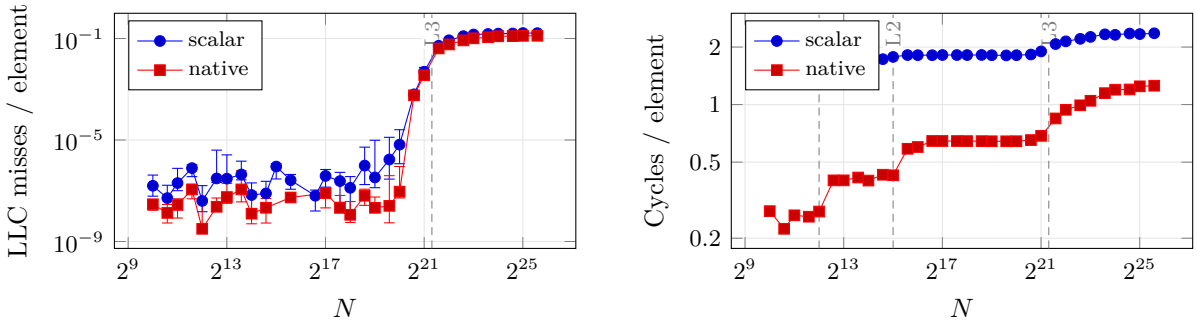
**Hypothesis (from first principles, then compared with the data)**

For float32 SAXPY (two arrays, 8 B/element) the per-element time should be flat and low while the working set fits in L1 ($2N \cdot 4\,\mathrm{B} \le 32\,\mathrm{KiB}$, i.e. $N \lesssim 4096$), step up at the L2 boundary ($N \approx 32\,768$) and again past L3 ($N \approx 2.6 \times 10^6$), and saturate in DRAM. The scalar and SIMD curves should be far apart in L1 and converge once DRAM bandwidth, which both share, dominates.

| | |
|---|---|
| **Fixed** | kernel SAXPY; float32; unit stride; aligned; pinned to CPU 2 |
| **Varied** | $N$ (dense grid, powers of two and midpoints, across L1/L2/LLC/DRAM); variant |
| **Counters** | user-space cycles, instructions, LLC and L1D read misses (Section 2.5) |
| **Bandwidth** | elements/s $\times$ 12 B (read $x$, read and write $y$) vs. each level's triad roof |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh locality` |

## 6.1 Analysis

The transitions line up with the cache sizes. In L1 native SAXPY takes 0.080 ns to 0.100 ns per element, 5.6–6.9$\times$ faster than scalar; the speedup drops to 4.0–4.2$\times$ past L1, 3.1$\times$ past L2 and




(a) LLC misses per element.

(b) Core cycles per element.

Figure 5: User-space hardware counters for the locality sweep (`perf_event_open`).

2.8× on the L3 plateau. Past L3 the gap narrows but does *not* close, contradicting the hypothesis: native takes 0.43 ns to 0.45 ns and scalar 0.83 ns to 0.85 ns, a speedup of 1.88–1.96×.

In bandwidth terms (Table 6 and Figure 4) native SAXPY reaches 76, 86, 100 and 103 % of the measured triad roof in L1, L2, L3 and DRAM: bandwidth-bound in L3 and DRAM (in DRAM still below the 25.7 GiB/s of the copy stream, which has the same two-reads-one-write pattern) and issue-bound in L1 and L2. Scalar is flat at 17 GiB/s to 20.5 GiB/s through L3, far below every roof.

*Counters* (Figure 5). For DRAM-resident native SAXPY, LLC misses reach 0.122 to 0.128 per element, the compulsory $2/16 = 0.125$ (one line each of $x$ and $y$ per 16 elements), so the vector loop moves no more data than it must. Scalar shows 0.156 to 0.164, about 28 % more, for a reason we have not established (Section 12). The cycle counts explain the remaining gap:

| cycles/element (IPC) | L1 ($N = 2048$) | L2 ($N = 16\,384$) | L3 ($N = 524\,288$) | DRAM ($N = 33\,554\,432$) |
|---|---|---|---|---|
| scalar | 1.53 (4.6) | 1.68 (4.2) | 1.81 (3.9) | 2.34 (3.0) |
| native | 0.26 (3.1) | 0.40 (1.9) | 0.64 (1.2) | 1.25 (0.6) |

Native SAXPY's cycles per element grow 4.7× from L1 to DRAM and its IPC falls to 0.6: it waits on memory. Scalar already spends 1.53 cycles per element in L1 at IPC 4.6, this core's issue limit, and in DRAM still retires 3.0 instructions per cycle: most of its time is execution. $2.34/1.25 = 1.88$ is exactly the measured DRAM speedup at this size. So once native sits on the bandwidth roof, the speedup is scalar's instruction-bound time over the memory time, a ratio that depends on the clock (Section 2).

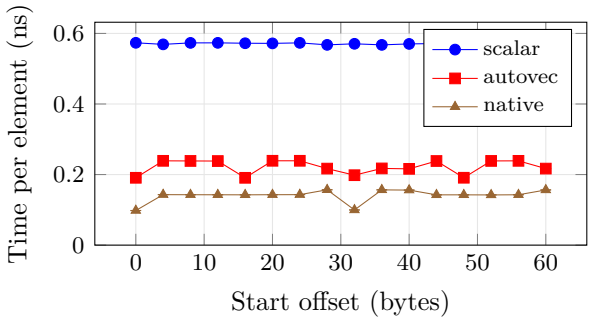
# 7 Alignment and Tail Handling

**Hypothesis (from first principles, then compared with the data)**

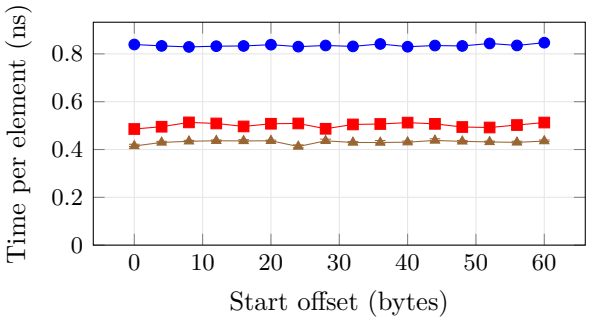
A start address that is not 32 B-aligned should make some 256 bit loads unaligned and/or trigger a scalar peel prologue, a modest fixed cost most visible at small $N$. On a client Skylake core unaligned `vmovups` within one cache line are nearly free, so we expect a small penalty (tens of percent) that recurs with 64 B period across a line.

**Hypothesis (from first principles, then compared with the data)**

(For the extended sweep.) `scalar`'s 4 B loads never straddle a line, so it should be insensitive to the offset, and `autovec` (16 B loads) should suffer less than `native`. In DRAM the misalignment cost should vanish into memory stalls. For short loops the time per *call* should grow in steps of

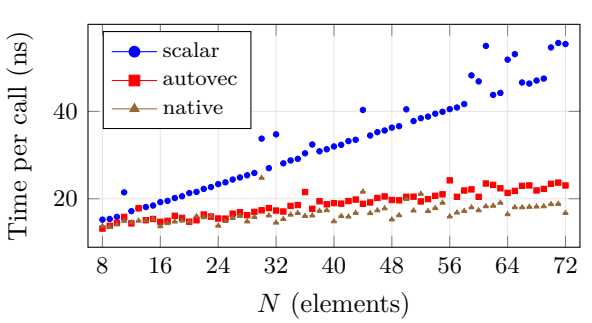



(a) $N = 4096$ (L1-resident).

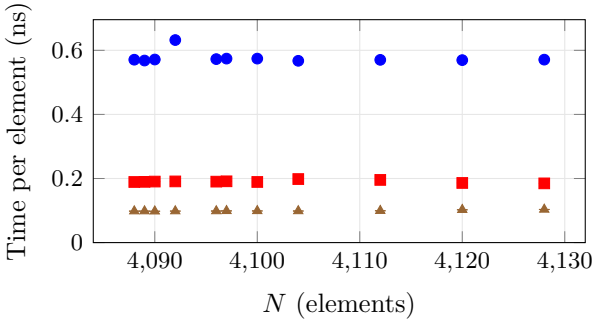



(b) $N = 16\,777\,216$ (DRAM-resident).

Figure 6: Cost of a misaligned start for SAXPY, by build variant (median, IQR error bars).




(a) Short loops: time per call, $N = 8 \ldots 72$ (IQR bars, mostly hidden by the marks).




(b) Across an 8-lane boundary near $N = 4096$.

Figure 7: Tail handling: remainder elements processed by the vectorized epilogue (offset 0).

8 elements (`native`) or 4 (`autovec`), with the vectorized epilogue making the remainder cheap but not free.

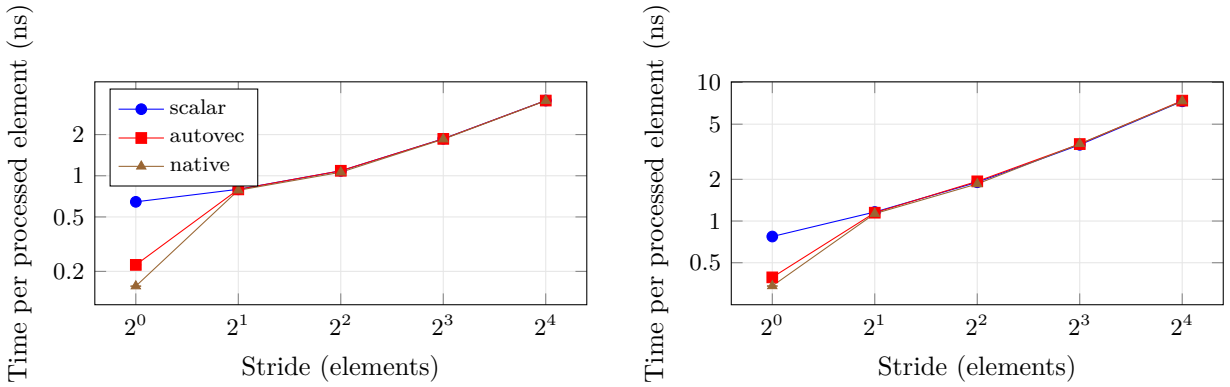
| | |
|---|---|
| **Fixed** | kernel SAXPY; float32; unit stride |
| **Varied** | start offsets $0, 4, 8, \ldots, 60$ B (every multiple of 4) at $N = 4096$ (L1-resident, isolates the prologue from bandwidth) and $N = 16\,777\,216$ (128 MiB, DRAM-resident); tail: every $N$ from 8 to 72 and $N$ stepping across an 8-lane boundary near 4096, offset 0; all three build variants |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh alignment` |

## 7.1 Analysis

*Misaligned starts, L1-resident* (Figure 6a). Native SAXPY runs at 0.098 ns to 0.100 ns per element at the 32 B-aligned offsets 0 and 32, 46 % slower at most other offsets and up to 60 % slower at 28, 36, 40 and 60. `autovec` is fast exactly at multiples of 16 and 13 % to 25 % slower elsewhere; `scalar` varies by only 1.0 %. This is what cache-line splits predict: off a 32-byte boundary every second 32 B load straddles a 64 B line, every fourth 16 B load does, and a 4 B load never does; GCC does not peel to an aligned start, so every split pays. The second level at 28, 36, 40 and 60 is unexplained (untested: split stores to `y` interacting with loads of `x`). Every IQR is below 0.3 %.

*DRAM-resident* (Figure 6b). The spread across offsets is only 6.1 % (native), 5.7 % (autovec) and 2.1 % (scalar), with no split-load pattern: the cost hides under DRAM misses, as hypothesized.

*Tails* (Figure 7). Near $N = 4096$ the 1 to 7 leftover elements are invisible. In short loops




(a) $N = 32\,768$ (cache-resident span).

(b) $N = 4\,194\,304$ (DRAM-resident).

Figure 8: Effect of stride on SAXPY time per *processed* element (median, IQR error bars). The three builds coincide for every non-unit stride.

($N = 8 \ldots 72$) native's time per call is lowest at multiples of 8 (13.7 ns at $N = 16$ to 16.7 ns at $N = 72$) and typically 2 ns to 3 ns higher at $N \equiv 7 \pmod 8$: the sawtooth of one 128 bit step plus up to three scalar iterations (Section 4). A fixed $\approx 10$ ns to 13 ns per call (call, setup, aliasing check) means native beats scalar by only 1.11× at $N = 8$ and 3.32× at $N = 72$. A masked AVX2 tail (`vmaskmovps`) would replace the remainder path; we did not implement one.

## 8 Stride and Gather Access

**Hypothesis (from first principles, then compared with the data)**

Unit stride lets each 64 B line serve 16 float32 elements; at stride 16 each element needs its own line. Time per processed element should rise roughly linearly with stride until it saturates at one line per element, and the SIMD advantage should vanish for any non-unit stride, because lanes must be loaded individually. Gather should be the worst case.

**Hypothesis (from first principles, then compared with the data)**

(For the extended sweep.) The explicit AVX2 gather (`vgatherdps`, Section 4) still needs one line per element and eight scalar stores, so it should be at best modestly faster than the scalar gather in cache. This CPU runs the "Gather Data Sampling" microcode mitigation (`/sys/devices/system/cpu/vulnerabilities/gather_data_sampling`), reported to slow `vgatherdps` considerably, so it may even lose.

| | |
|---|---|
| **Fixed** | SAXPY; float32 |
| **Varied** | access (unit, stride 1/2/4/8/16, gather, explicit AVX2 gather); variant (scalar, autovec, native; AVX2 gather native only); $N = 32\,768$ processed elements (span 256 KiB × $s$: L2 for $s = 1$, L3 for $s = 16$) and $N = 4\,194\,304$ (DRAM for every $s$) |
| **Definition** | stride $s$ processes elements $0, s, 2s, \ldots$ over an array of $Ns$ elements; gather applies SAXPY through a random permutation index so every element is touched once with no spatial locality. |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh stride` |

Table 7: Time per processed element (ns) by access pattern. *AVX2 gather* is the explicit `vgatherdps` kernel (native build only); speedup is scalar over the faster native kernel (the AVX2 gather has no strided form, hence –).

| $N$ | Access | scalar | autovec | native | AVX2 gather | speedup (×) |
|---|---|---|---|---|---|---|
| 32768 | unit | 0.636 | 0.225 | 0.178 | – | 3.57 |
| 32768 | stride 2 | 0.796 | 0.795 | 0.783 | – | 1.02 |
| 32768 | stride 4 | 1.087 | 1.088 | 1.061 | – | 1.02 |
| 32768 | stride 8 | 1.865 | 1.862 | 1.851 | – | 1.01 |
| 32768 | stride 16 | 3.551 | 3.548 | 3.538 | – | 1.00 |
| 32768 | gather | 2.023 | 2.100 | 2.157 | 3.225 | 0.94 |
| 4194304 | unit | 0.784 | 0.370 | 0.334 | – | 2.35 |
| 4194304 | stride 2 | 1.162 | 1.149 | 1.131 | – | 1.03 |
| 4194304 | stride 4 | 1.905 | 1.934 | 1.852 | – | 1.03 |
| 4194304 | stride 8 | 3.549 | 3.594 | 3.611 | – | 0.98 |
| 4194304 | stride 16 | 7.313 | 7.396 | 7.336 | – | 1.00 |
| 4194304 | gather | 10.205 | 9.979 | 10.198 | 10.911 | 1.00 |

### 8.1 Analysis

Only unit stride lets `native` beat `scalar` (Table 7): 3.6–4.1× at $N = 32\,768$ and 2.35× in DRAM. The range comes from two runs of identical machine code in different processes (the table's *unit* row and the stride-1 point of Figure 8a; Section 12).

At stride 2 and beyond the builds agree within 0.98 to 1.03×, because every $s > 1$ runs the scalar copy of the versioned loop (Section 4), and cost follows the cache lines touched. In DRAM native time per element grows $1.13 \to 1.85 \to 3.61 \to 7.34$ ns from stride 2 to 16. Counting fetched traffic as $3 \times \min(4s, 64)$ B per element (`stride_bw.csv`), useful bandwidth falls from 9.9 to 1.5 GiB/s while fetched traffic stays at 24.1 to 24.8 GiB/s from stride 4 on, the DRAM triad roof: memory is saturated with lines of which only $4s/64$ is used. Gather is the worst pattern: 10.2 ns per element in DRAM, 31× unit stride, because random order also defeats the prefetcher.

*Hardware gather.* The explicit AVX2 kernel takes 1.59× (3.23 ns, $N = 32\,768$) and 1.07× (10.91 ns, DRAM) the time of the scalar build's gather, as the second hypothesis anticipated: two mitigated `vgatherdps` plus eight scalar stores per step. Testing without the mitigation needs a reboot as root. Access pattern, not vector width, is the first-order knob for these kernels.

## 9 Data Type Comparison

**Hypothesis (from first principles, then compared with the data)**

A 256 bit register holds 8 float32 but only 4 float64 lanes, so float64 should show roughly half the SIMD speedup of float32 for the same kernel while data is cache-resident. int32 elementwise multiply should behave like float32 (8 lanes, one `vpmulld`). In DRAM, float64 moves twice the bytes per element, so its *absolute* throughput should be lower and its speedup should compress toward 1× just as float32 does.

| | |
|---|---|
| **Fixed** | unit stride; aligned |
| **Varied** | data type (float32, float64, int32); kernel; variant; $N = 2^{10} \dots 2^{24}$ |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh dtype` |

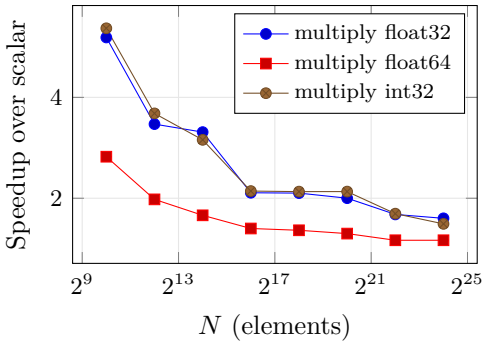



Figure 9: Native speedup of element-wise multiply by data type (median; error bars: range implied by the two IQRs). float32 and int32 (8 lanes) track each other; float64 (4 lanes) is about half. SAXPY and stencil: Table 8.

Table 8: Speedup over scalar by kernel and data type (`dtype.csv`), at $N = 1024$ (L1), 16 384 (L2) and 16 777 216 (DRAM).

| Kernel | Type | autovec, 1024 | native, 1024 | 16 384 | 16 777 216 |
|---|---|---|---|---|---|
| SAXPY | float32 | 2.84 | 5.60 | 4.08 | 1.99 |
| SAXPY | float64 | 1.65 | 3.28 | 2.21 | 1.38 |
| multiply | float32 | 3.53 | 5.19 | 3.31 | 1.60 |
| multiply | float64 | 1.91 | 2.82 | 1.66 | 1.17 |
| multiply | int32 | 1.86 | 5.37 | 3.16 | 1.49 |
| stencil | float32 | 2.75 | 3.35 | 3.44 | 2.79 |
| stencil | float64 | 1.44 | 1.73 | 3.02 | 1.68 |

### 9.1 Analysis

At $N = 1024$ native float64 keeps 0.52 to 0.59 of float32's speedup for all three kernels (multiply 2.82× vs 5.19×, SAXPY 3.28× vs 5.60×, stencil 1.73× vs 3.35×), close to the predicted 1/2, and int32 multiply (one `vpmulld` over 8 lanes) lands on the float32 curve (5.37×). The ratio is not universal: at $N = 16\,384$ the stencil keeps 3.02× in float64 against 3.44× in float32, because the float32 stencil is depressed at these sizes (Section 12), and by $N = 65\,536$ multiply narrows to 1.40× vs 2.11×.

*Operation cost.* `autovec` gains 3.53× on float32 multiply at $N = 1024$ but only 1.86× on int32: SSE2 has no packed 32-bit multiply (`pmulld` is SSE4.1), so GCC emulates it with seven instructions per 4 lanes (two `pmuludq`, two shifts, two `pshufd` and a `punpckldq`).

In DRAM ($N = 16\,777\,216$) every speedup compresses as predicted (native float32 1.60–2.79×, float64 1.17–1.68×), and float64 keeps 60 % to 73 % of the float32 gain. Float32 multiply's 1.51× in the baseline sweep differs by a 6 % run-to-run spread in native throughput, so we quote 1.5–1.6×. Float64's absolute multiply throughput is under half of float32's (0.80 vs 1.72 GFLOP/s) because each element moves twice the bytes. Lane count and operation cost set the cache-resident speedup; bytes per element set the DRAM throughput.

## 10 Roofline Interpretation

Table 9: Measured roofline ceilings, single core (`run.sh ceilings`). Top: register-only accumulator loops; FLOP/cycle and clock from the cycle counter. Bottom: STREAM-style bandwidth (GiB/s) at one footprint per level; "incl. WA" adds the write-allocate read. Roofs use the explicit triad in L1 and triad incl. WA below it; the read stream (*) is add-latency-bound and not a roof.

| Peak loop | GFLOP/s | FLOP/cycle | GHz |
|---|---|---|---|
| scalar_f32 | 5.6 | 1.99 | 2.79 |
| scalar_fma_f32 | 11.2 | 4.00 | 2.79 |
| sse_f32 | 22.3 | 8.00 | 2.79 |
| sse_f64 | 11.1 | 3.98 | 2.79 |
| avx2_fma_f32 | 89.2 | 31.99 | 2.79 |
| avx2_fma_f64 | 44.6 | 16.00 | 2.79 |

| | | GiB/s | | | |
|---|---|---|---|---|---|
| Level | Footprint (B) | read* | copy | triad | triad incl. WA |
| L1 | 16,128 | 85.4 | 142.9 | 155.6 | 207.5 |
| L2 | 130,944 | 71.3 | 67.9 | 67.9 | 90.5 |
| L3 | 8,388,480 | 39.3 | 33.4 | 36.4 | 48.6 |
| DRAM | 536,870,784 | 16.1 | 17.1 | 18.2 | 24.3 |

Table 10: Arithmetic intensity (float32) from the actual traffic per element, the same convention as the triad roof: L2, L3 and DRAM include the write-allocate read of a store to an unread line; in L1 every store hits, so there is none.

| Kernel | $I$, L2–DRAM | $I$, L1 | Traffic per element (L2–DRAM) |
|---|---|---|---|
| SAXPY | 0.167 | 0.167 | 2 FLOP/12 B: read $x, y$, write $y$ (already read: no write-allocate) |
| multiply | 0.063 | 0.083 | 1 FLOP/16 B: read $x, y$, write $z$ + write-allocate of $z$ |
| stencil | 0.417 | 0.625 | 5 FLOP/12 B: read $x$ once (reuse), write $y$ + write-allocate |
| dot product | 0.25 | 0.25 | 2 FLOP/8 B: read $x, y$, no store |
| triad (roof) | 0.125 | 0.167 | 2 FLOP/16 B: read $b, c$, write $a$ + write-allocate |

**Hypothesis (from first principles, then compared with the data)**

All four kernels have very low arithmetic intensity (well under 1 FLOP/B). With single-core bandwidth of tens of GB/s and a peak vector rate near 100 GFLOP/s, all should sit on the *bandwidth* slope when DRAM-resident. The stencil, with the highest intensity, should come closest to the compute roof.

**Hypothesis (from first principles, then compared with the data)**

(For the measured ceilings.) Even the L1 roof should put the ridge point above 0.5 FLOP/B, so cache-resident native kernels should also be bounded by bandwidth (or the load/store ports behind it), not by the FMA units.

Attainable performance at arithmetic intensity $I$ (FLOPs per byte of memory traffic) is bounded by

$$P(I) = \min\big(P_{\text{peak}},\ B_\ell \cdot I\big), \tag{1}$$

where $B_\ell$ is the bandwidth of the memory level $\ell$ holding the data [1]. We draw one roof per level (a cache-aware roofline); both ceilings come from separate microbenchmarks (Section 2.5), never from the kernels plotted.

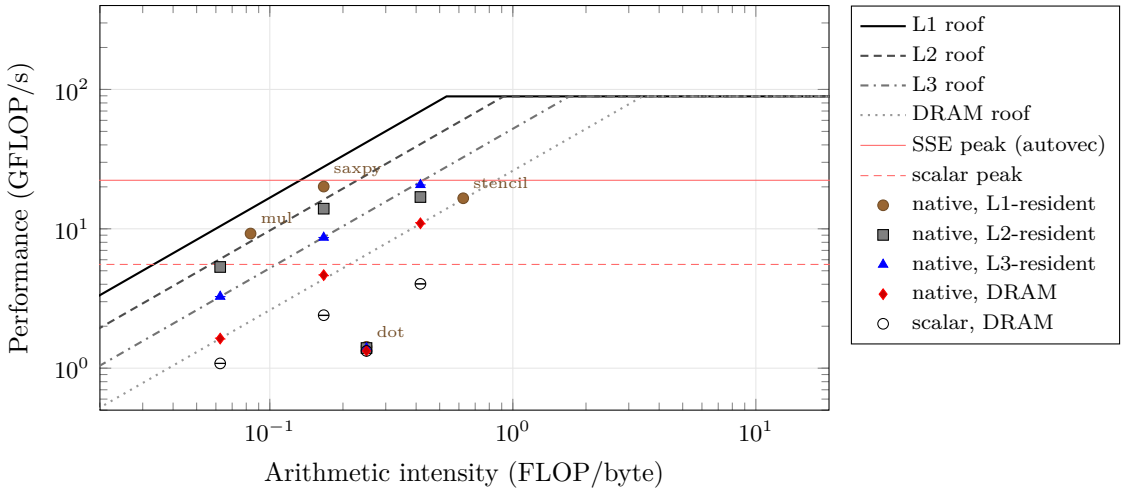



Figure 10: Cache-aware roofline for one core. Roofs: measured triad bandwidth per level, capped by the measured AVX2-FMA peak; thin red lines: measured SSE and scalar peaks. Points: native (and DRAM scalar) GFLOP/s from Table 5, L1 points at their L1 intensity; IQR bars mostly hidden by the markers.

### 10.1 Analysis

*Ceilings* (Table 9). The AVX2 FMA loop reaches 89.2 GFLOP/s float32 (44.6 float64), 31.99 FLOP/cycle at 2.79 GHz: exactly the 2 FMA ports × 8 lanes × 2 = 32 the core allows. The SSE and scalar loops also reach their port limits (8.00 and 1.99 FLOP/cycle). The ridge points $P_{\text{peak}}/B_\ell$ are ≈ 0.53, 0.92, 1.71 and 3.42 FLOP/B from L1 to DRAM; the L1 ridge is just above the 0.5 FLOP/B of the second hypothesis, and only the L1 stencil ($I = 0.625$) lies right of it.

*DRAM-resident kernels.* Every intensity (0.06 to 0.42 FLOP/byte) is far left of the DRAM ridge, so all four kernels are bandwidth-bound, as hypothesized. Native multiply reaches 100 % of the DRAM roof, the stencil 101 % and SAXPY 107 % (4.65 vs 4.35 GFLOP/s). Sustainable bandwidth depends a little on the access pattern (SAXPY writes a line it has just read), so the triad roof is a reference, not a bound to the byte; since it comes from a different loop, landing on it is evidence, not a tautology. The scalar points sit at 37 % to 66 % of the roof (dot: 20 %): the instruction-bound loops of Section 6.

*Cache-resident kernels.* L3-resident native points sit at 95 % to 100 % of the L3 roof. In L2 SAXPY and multiply reach 86 % and 88 % (stencil 42 %); in L1 SAXPY reaches 72 % of the roof and 23 % of the FMA peak, and the stencil 19 % of the peak that bounds it. So, confirming the second hypothesis, cache-resident kernels are bounded by load/store and instruction issue, not the FMA units. From L2 outward each step in Figure 3 is a move to a lower roof.

*The dot product* is far *below* every roof because it is latency-bound: one scalar `vaddss` per element with a 4-cycle latency allows 0.5 FLOP/cycle, 1.39 GFLOP/s, and it measures 1.39 GFLOP/s to 1.42 GFLOP/s for $N = 1024$ to 1 048 576. Wider vectors cannot help a kernel waiting on memory; only higher intensity (fusing passes, cache blocking) can.

# 11 Negative Example

**Hypothesis (from first principles, then compared with the data)**

A naive benchmark—or a quickly generated one that only ever tests a small array—would conclude that SAXPY enjoys the full $\approx 6\times$ SIMD speedup. We predict instead that the same *vectorized* kernel shows almost no speedup on a DRAM-resident array, because memory bandwidth, not vector throughput, then sets the pace.

| | |
|---|---|
| **Case** | memory-bound SAXPY: a loop that provably vectorizes (`vfmadd213ps` on `ymm`, Section 4) but whose working set far exceeds L3 |
| **Build** | `native` (`-O3 -march=native`) vs `scalar` |
| **Script** | `SIMD_Advantage_Profiling/scripts/run.sh negative` |

## 11.1 Evidence

The SAXPY loop at `kernels.c:29` is vectorized at 256 bit with FMA (Section 4), yet its speedup over scalar falls with size:

| $N$ | working set | native GFLOP/s | % of roof | speedup |
|---|---|---|---|---|
| 1024 | 8 KiB (L1) | 20.1 | 72 | 5.60× |
| 16 384 | 128 KiB (L2) | 13.7 | 84 | 4.11× |
| 262 144 | 2 MiB (L3) | 8.6 | 99 | 2.82× |
| 4 194 304 | 32 MiB (DRAM) | 6.0 | – | 2.31× |
| 8 388 608 | 64 MiB (DRAM) | 5.2 | 120 | 2.19× |
| 16 777 216 | 128 MiB (DRAM) | 4.7 | 109 | 1.95× |
| 33 554 432 | 256 MiB (DRAM) | 4.5 | 104 | 1.92× |

"% of roof": native GFLOP/s over $B_\ell \cdot I$ for the level holding the data (Section 10); 32 MiB straddles L3 and DRAM.

## 11.2 Explanation

The hypothesis was half right: the $\approx 6\times$ of L1 does not survive in DRAM, but the speedup settles at 1.92–1.95×, not "almost none". The loop genuinely uses SIMD, but at 0.167 FLOP/B it is deep in the bandwidth-bound region: native SAXPY runs at 104 % to 120 % of the separately measured DRAM roof with the compulsory 0.122 to 0.128 LLC misses per element (Section 6). *No* wider vector, AVX-512 included, could make it faster. The remaining $\approx 1.9\times$ is the scalar loop's instruction cost (2.34 vs 1.25 cycles per element), which shrinks at turbo clocks (Section 2).

The dot product is subtler: it reports "vectorized" and uses a 256 bit `vmulps`, yet the serial reduction leaves $\approx 1\times$. Both carry the same lesson for AI-assisted benchmarking: "it compiled to SIMD" is necessary but not sufficient evidence that a kernel is faster.

# 12 Anomalies and Limitations

## 12.1 Unexpected results

The dot product, the DRAM gap and the losing AVX2 gather are explained in Sections 4, 6 and 8. Three anomalies remain open:

- **Cache-resident stencil is slower than L3-resident stencil**: 16.6 GFLOP/s to 17.3 GFLOP/s for $N = 1024 \ldots 16\,384$ but 19.9 GFLOP/s to 20.7 GFLOP/s for $N = 65\,536 \ldots 1\,048\,576$, reproducibly, which no monotonic cache effect explains. Untested hypothesis: 4K aliasing between stores to `y[i]` and loads of `x[i+1]`; sweeping the `x`–`y` distance modulo 4 KiB with the `ld_blocks_partial.address_alias` counter would test it.
- **Native SAXPY peaks within L1** (6.86× at $N = 1536$, 6.32× at $N = 512$, else 5.6–5.9×); the same placement hypothesis applies.
- **Same code, different process,** $N = 32\,768$: 1 % (scalar) and 14 % (native) apart. Untested hypothesis: the two arrays exactly fill the 256 KiB L2, so conflict misses depend on physical page placement.

### 12.2 Limitations

- **Core-PMU counters only**: no DRAM-traffic or TLB events, and the generic LLC-miss event's prefetch handling is model-specific, so we only compare it between builds. IMC bytes per element (as read by the companion Cache & Memory report) would turn the roofline's traffic assumption into a measurement.
- **Fixed clock**: reproducible but not normal operation; at turbo, DRAM-resident SAXPY speedups shrink to 1.32–1.44× (Section 2).
- **One machine, one compiler** (i9-10900F, GCC gcc (GCC) 16.2.1 20260810), no AVX-512 part.
- **THP always-on, no core isolation, unshuffled sweep order**; the shared-core checks of Section 2 guard the data.
- **Discarded sessions**: earlier sessions that day, before the GPE storm was masked and while another agent's unlocked smoke tests briefly shared CPU 2.

## 13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

**Tools used**

Claude Code, an AI coding agent, for coding, debugging, and running the experiments. All sessions used the model Claude Opus 5.5. On 2026-09-15 it produced the templates (folder structure, READMEs, Makefiles, files with placeholder markers, report skeletons) that every later step filled in. On 2026-10-12 to 2026-10-20 it produced the implementation, and on 2026-10-21 the rework (measured ceilings, counters, AVX2 gather, extended sweeps and the fixed-clock re-run). It also produced a preliminary draft of the report, which I discarded; I wrote this report myself. GCC gcc (GCC) 16.2.1 20260810 and `objdump` produced the builds and the vectorization evidence.

**Who ran the measurements**

The timed SIMD experiments of 2026-10-21 (the reported runs, plus earlier sessions that were discarded for interference) were executed by Claude Code on my machine, with my approval. Before the runs I set the governor to `performance`, disabled turbo and SMT, and masked the

ACPI GPE 0x42 interrupt storm. Claude Code checked these settings before each run and serialized its runs with other benchmark jobs through a lock file. All measurements come from these real runs on the machine in Table 1.

**What they helped generate**

The four kernels and their references; the build-variant Makefile; the benchmark driver and the shared timer/allocation/pinning helpers; the experiment scripts (`run.sh`), the summarizer (`summarize.c`) and the vectorization-evidence script; the roofline-ceiling microbenchmarks (`ceilings.c`), the `perf_event_open` counter helper (`perfctr.c`), the explicit AVX2 gather kernel and the post-processing scripts (`speedup_iqr.awk`, `roofline.sh`, `tables.sh`).

**Who verified what**

The AI ran the automated checks: `make test` (578 cases in total), the disassembly comparisons, the per-run settings and shared-core checks, and the CSV-to-text spot checks of every quoted number. I verified all AI output, applied the root-only system settings, and reviewed the results, which drove several of the corrections below. "Independently" therefore means checked against measurements and tool output, as well as against my own analysis.

**What was independently modified, tested, or rejected**

Concrete corrections, among others. (1) The generated Makefile compiled the *whole* program with each variant's flags; this was restructured so only the kernel translation unit varies, keeping the timing harness identical across builds. (2) The processed-CSV column names and one figure filter had drifted from what the plots expect—the stride figure asked for `n=16777216` while the data was measured at `n=4194304`—which would have produced a silently empty plot; the filter was corrected to the measured size. (3) *An AI-written script was simply wrong*: the `awk` in `verify_vectorization.sh` took its output path as a trailing `VAR=value` argument, which `awk` does not assign until it reaches that argument, so the `BEGIN` block saw an empty string and aborted with a null-redirection error. It was caught the first time `make report` was run and fixed by passing the path with `-v`. We also chose *not* to enable `-ffast-math`, rejecting the easy way to make the dot product vectorize, because it would break bit-reproducibility against the reference—and because the non-reassociated reduction is itself a result worth reporting (Sections 4 and 11).

**Validation**

Correctness: 578 reference and known-answer cases in total (190 scalar, 190 autovec, 198 native) pass (Section 3). Vectorization: confirmed by compiler report and disassembly, not assumed (Section 4). Numbers: every number from the 2026-10-21 runs traces to a committed file (Section A); the memory-bound claims are cross-checked against bandwidth and peak FLOP rate measured by separate microbenchmarks (`src/ceilings.c`), not derived from the kernels themselves (Section 10). Review corrections that changed results: an early roofline bandwidth was derived from SAXPY itself (circular; replaced by the independent ceilings); the AI-drafted claims that scalar and SIMD "converge" in DRAM and that small-$N$ speedups are inflated by per-call overhead were both contradicted by the fixed-clock data and ignored; the alignment sweep had silently skipped offsets 40 B to 56 B and was re-run; and the L1 roof first included a write-allocate read that L1-resident stores do not incur.

# 14 Conclusion

On this AVX2 machine SIMD pays off handsomely only when the data fits in cache and the kernel is not a serial reduction: `-march=native` then gives 3.5–5.8× at $N = 4096$, limited by load/store issue rather than the FMA units. In DRAM the vector builds run *at* the independently measured bandwidth roof, and what remains (1.88–2.02× for SAXPY) is the scalar loops' own instruction cost at the fixed clock. Float64 keeps about half the cache-resident gain, and any non-unit stride or gather erases it. The main lesson: "the loop vectorized" is not "the loop is fast".

# References


[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785


# A Reproducibility

Code, data (`ACS-Code/`) and reports (`ACS-Latex/`) live in one repository. The timed runs (2026-10-21, 20:38–20:54) used the tree committed as `b00db66`; the measurement code is unchanged since, except an opt-in error print in the validation path of `main.c` that is inactive in timed runs. This report corresponds to commit `65219fa68aef`. From the repository root:

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout 65219fa68aef
cd ACS-Code
make tools                                  # tools/summarize
make -C SIMD_Advantage_Profiling            # scalar/autovec/native simd_bench + ceilings
make -C SIMD_Advantage_Profiling test       # 578 correctness cases (max error logged)
make -C SIMD_Advantage_Profiling report     # vectorization evidence
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
SIMD_Advantage_Profiling/scripts/run.sh all     # ceilings, baseline, locality, alignment,
                                                # stride, dtype, negative
scripts/sync_data.sh SIMD_Advantage_Profiling   # -> ACS-Latex/SIMD_Advantage_Profiling/data/
cd ../ACS-Latex && make simd                    # -> SIMD_Advantage_Profiling.pdf
```

Before timing, as root: governor `performance`, `intel_pstate/no_turbo` = 1, SMT off, and `echo disable > /sys/firmware/acpi/interrupts/gpe42` (none persists across a reboot). `run.sh derive` recomputes the derived tables without measuring. Each raw directory (Table 11) holds `raw.csv` and `env.txt`; the turbo comparison traces to commit `73f8fb8`.

Table 11: Raw data behind each figure and table.

| Figure / table | Raw run directory (under `ACS-Code/SIMD_Advantage_Profiling/data/raw/`) |
|---|---|
| Figures 1 and 2 | `baseline/20261021-203854` |
| Figures 3 to 5, Table 6 | `locality/20261021-204801` |
| Figures 6 and 7 | `alignment/20261021-204858` (`raw.csv`, `raw_tail.csv`) |
| Figure 8, Table 7 | `stride/20261021-205143` |
| Figure 9, Table 8 | `dtype/20261021-205209` |
| Table 5 | `baseline/20261021-203854` |
| Correctness result (Section 3) | `validation/20261021-231629` (`make_test.txt`) |
| Figure 10, Table 9 | `ceilings/20261021-204708`, `baseline/20261021-203854` |
| Negative-case table (Section 11) | `negative/20261021-205359` |
| Table 4 | `vectorization/20261021-001032` |